

Example 1:

```
#include <iostream>
#include <unordered_map>
#include <string>
using namespace std;
```

```
// Define possible data types
enum Type { INT, FLOAT, STRING, Unknown }
```

```
// Symbol Table class
```

```
class SymbolTable {
    unordered_map<string, Type> table; // stores variable name & type
```

```
public:
```

```
    // Insert a variable into table
```

```
    void insert (string name, Type t) {
        if (table.find(name) != table.end())
            cout << "Error: Variable " << name << " already declared.\n";
```

```
    else
```

```
        table[name] = t; // store variable
    }
```



```
//Lookup variable type
Type lookup(string name) {
    if (table.find(name) != table.end())
        return return table[name];
    cout << "Error: Variable " << name << " not declared.\n";
    return UNKNOWN;
}
```

```
//Print Symbol table
void print() {
    for(auto & p : table) {
        string typeStr = (p.second == INT)? "int":
                          (p.second == FLOAT)? "float": "string";
        cout << p.first << ":" << typeStr << endl;
    }
};
```

```
int main() {
    SymbolTable sym;
    sym.insert("x", INT); // insert variable x
    sym.insert("y", FLOAT); // insert variable y
    sym.insert("x", INT); // duplicate -> error
    sym.print(); // display table
    sym.lookup("x"); // valid lookup
}
```

GIVENCHY


```
sym.lookup("z"); // undeclared -> error  
return 0;
```

}

Output:

Error: Variable 'x' already declared

x: int

y: float

Error: Variable 'z' not declared.

Example 2:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// Define types
```

```
enum Type {INT, FLOAT, Error};
```

```
// Expression tree node
```

```
struct ExprNode {
```

```
    Type type;
```

```
    string op; // operators (+, -, *, /)
```

```
    ExprNode *left, *right;
```

```
    int intVal; // integer value
```

GIVENCHY


```
float floatVal; // float value  
};
```

```
// function to check expression types  
Type checkExpr (ExprNode* node) {  
    if (!node) return Error;  
    // leaf node (constant)  
    if (node->op.empty()) {  
        return node->type;  
    }  
}
```

```
// Arithmetic rules
```

```
if (node->op == "+" || node->op == "-" ||  
    node->op == "*" || node->op == "/") {  
    // if any operand is float -> result is float  
    if (leftType == float || rightType == float)  
        return float;
```

```
// Both int -> result int  
else if (leftType == INT && rightType == INT)  
    return INT; }
```

```
cout << "Type error: incompatible type";  
return error; }
```

```
int main () {
```

```
    // Build expression: (2 + 3.5) * 4
```


ExprNode* leaf2 = new ExprNode{INT, "", nullptr,
nullptr, 2, 0};

ExprNode* leaf3-5 = new ExprNode{Float, "",
nullptr, nullptr, 0, 3.5};

// (2+3.5)

ExprNode* plus = new ExprNode{ERROR, "+",
leaf2, leaf3-5, 0, 0};

ExprNode* leaf4 = new ExprNode{INT, "",
nullptr, nullptr, 4, 0};

// (2+3.5)*4

ExprNode* mult = new ExprNode{ERROR,
"*", plus, leaf4, 0, 0};

// check type

Type result = checkExpr(mult);

if (result == INT) cout << "Expression type: int";

else if (result == Float) cout << "Expr type: float";

else cout << "Type error\n";

return 0;

}

Output:

Expression type: float

Example 3:

```
#include <iostream>
using namespace std;
// Define new type names
typedef int Age;
typedef int Weight;
// Two structures with same fields
struct Point {int x, y;};
struct Coordinate {int x, y;};
int main () {
    Age a = 25;
    Weight w = 70;
    // Name equivalence:
    // Age and Weight are treated as diff
    cout << "Age: " << a << " , Weight: " << w << endl;
    // structure
    point p = {1, 2};
    // This would be an error
    // Coordinate c = p;
    // Structural equivalence check (manual)
    bool sameStructure = (sizeof(Point) == sizeof
        (Coordinate));
```



```

cout << "Same Structure?" << sameStructure
    << endl;
cout << "But treated as diff type in C++";
return 0;
}

```

Output:

Age: 25, Weight: 70
 Point and Coordination have same size?
 But C++ use name equivalence, so they are
 incompatible

Example 4:

```

#include <iostream>
#include <string>
#include <unordered_map>
using namespace std;
// Define types
enum Type { INT, FLOAT, STRING, UNKNOWN };
// Type Checker class
class TypeChecker {
    unordered_map<string, Type> symTable;
public

```


// Declare Variable

```
void declare (string name, Type t) {  
    if (symTable.find(name) != symTable.end()  
        ( ))
```

```
        cout << "Error: " << name << " already declared;  
    else
```

```
        symTable[name] = t;  
}
```

// check Assignment

```
void check checkAssignment (string lhs, string rhs,  
                             Type lhsType) {
```

```
    Type * lhsType = getType (lhs);  
    if (lhsType == UNKNOWN || rhsType == UNKNOWN)  
        return;
```

```
    if (lhsType != rhsType) {  
        cout << "Type mismatch"; }  
    else {
```

```
        cout << "OK:" << lhs << "=" << rhs << ;  
    }  
}
```

```
int main () {
```

```
    TypeChecker tc;
```


// Declarations

tc.declare ("x", INT);

tc.declare ("y", FLOAT);

tc.declare ("name", STRING);

// Assignment

tc.checkAssignment ("x", "5", INT); //OK

tc.checkAssignment ("y", "3.14", FLOAT); //OK

tc.checkAssignment ("x", "hello", STRING); //Error

tc.checkAssignment ("z", "10", INT); //undeclared

return 0;

}

Output:

OK: x = 5

OK: y = 3.14

Type mismatch: cannot assign string to int

Semantic error: z not declared!

Type mismatch: cannot assign int to
unknown